

Session Planner

Internes Coach-Tool zum Planen von Sessions und Zuweisen an Coaches

Der **Session Planner** hilft neue-fische-Coaches, ihre **Cohorten** und einzelnen **Sessions (Shifts)** zu verwalten und Shifts gezielt einzelnen Coaches zuzuweisen. Herzstück ist die **Schedule-View**: eine wählbare Woche (Mo–Fr) zeigt alle Shifts in einem Raster — zunächst in einer „Unassigned“-Spur, per Dropdown werden sie einem Coach zugewiesen und wandern in dessen Spur.

BACKEND

Java · Spring Boot · REST

DATENBANK

PostgreSQL · Spring Data JPA

FRONTEND

React · TypeScript · Vite

FRONTEND-LIBS

SWR · axios · React Hook Form · Styled Components

AUTH

GitHub OAuth2 (Provisioning-by-githubName)

QUALITÄT

Pflicht-Tests (Testcontainers) · SonarQube

DEPLOYMENT

Docker · Docker Hub · render.com (Web Service + Postgres)

ZEITRAHMEN

4 Wochen · MVP ≈ 3 Wochen

Datenmodell

User

name · role (MVP: ADMIN) · githubName (unique, Login-Matching) · optional email/avatar

Cohort

name · startDate · endDate · federalState (Bundesland) · courseName

Shift

date · startTime · endTime · title · cohort (→Cohort) · coach (→User, nullable)

Hinweis: Die Entity-Felder werden im Verlauf noch erweitert; diese Übersicht bildet den aktuellen Planungsstand ab.

MVP-Tickets

FUNDAMENT

SP-1 ~2 Tage

Projekt-Setup & Single-Deployable

Ziel: Lauffähiges Grundgerüst mit einem Deployable-Artefakt.

- Repo mit getrennten Ordnern backend/ (Spring Boot, Maven) und frontend/ (Vite + TS)
- Frontend-Build **manuell** per `npm run build` → Output nach `backend/src/main/resources/static/`
- PostgreSQL lokal + `application.properties`; Dev-Proxy Vite → :8080
- Feature-Package-Struktur (auth/user/cohort/shift/config)

DoD: `mvn package` erzeugt ein JAR, das API und die gebaute UI ausliefert.

SP-2 ~1 Tag

CI: Tests, SonarQube & Coverage

Ziel: Automatische Qualitätssicherung ab dem ersten Commit.

- GitHub-Actions-Pipeline (Build + Test bei jedem Push/PR)
- JaCoCo-Coverage-Report
- SonarQube/SonarCloud-Analyse eingebunden

DoD: Pipeline grün, Coverage & Sonar-Report sichtbar.

AUTH

SP-3 ~2,5 Tage

GitHub OAuth2 Login (Provisioning-by-githubName)

Ziel: Nur bekannte User dürfen sich einloggen.

- Spring Security OAuth2-Login über GitHub
- Custom `OAuth2UserService` : bestehenden User per `githubName` matchen — **kein Match** ⇒ **Zugriff verweigert** (kein Auto-Upsert)
- GET `/api/auth/me` , Logout, `/api/**` abgesichert
- CSRF via `CookieCsrfTokenRepository` + `axios xsrf-Config`
- Security-Tests (bekannter vs. unbekannter `githubName`)

DoD: Bekannter GitHub-User kommt rein, unbekannter wird abgelehnt.

CRUD-ENTITÄTEN

SP-4 ~1,5 Tage

User — Backend CRUD + Tests

Ziel: Vollständige User-API als Referenzmuster für die anderen Entitäten.

- Entity, Repository, Service, Controller, Request-/Response-DTOs (`record`)
- Endpunkte GET/POST/PUT/DELETE `/api/users` , Validierung, `@RestControllerAdvice`
- Controller- (MockMvc) + Service-Tests (Mockito)

DoD: CRUD durchtestbar, Tests grün.

SP-5 ~1,5 Tage

User — Frontend

Ziel: User-View mit dem wiederverwendbaren Formularmuster.

- Liste (Name, Rolle, GitHub-Name); Detail-Modal mit Edit/Delete; Create-Button
- **Ein** RHF-Formular für Create + Edit (Edit = vorbefüllt)
- SWR-Hook `useUsers` + zentrale `axios`-Instanz + `mutate()`

DoD: User anlegen, bearbeiten, löschen im UI.

SP-6 ~1 Tag

Cohort — Backend CRUD + Tests

Ziel: Cohort-API nach dem User-Muster.

- Entity/Repo/Service/Controller/DTOs für `name`, `startDate`, `endDate`, `federalState`, `courseName`
- Bundesland als Enum; Controller-/Service-Tests

DoD: CRUD + Tests grün.

SP-7 ~1 Tag

Cohort — Frontend

Ziel: Cohort-View analog zu Users.

- Liste + Detail-Modal + wiederverwendbares Formular (Datumfelder, Bundesland-Dropdown)
- SWR-Hook `useCohorts`

DoD: Cohorten im UI verwaltbar.

SP-8 ~2 Tage

Shift — Backend CRUD, Range-Query & Assign

Ziel: Shift-API inkl. der für Schedule nötigen Endpunkte.

- CRUD; Relationen `cohort` / `coach` als IDs in Requests, eingebettet in Responses
- GET `/api/shifts?from=&to=` (Wochen-Range)
- PUT `/api/shifts/{id}/coach` — `coachId` oder `null` (Unassign)
- Zeit-Validierung (`startTime < endTime`); Tests

DoD: CRUD, Range-Query & Assign inkl. null getestet.

SP-9 ~1,5 Tage

Shift — Frontend

Ziel: Shift-View mit Cohort- und Coach-Auswahl.

- Liste (Datum, von/bis, Titel, Cohort, Coach) + Detail-Modal
- Wiederverwendbares Formular mit Cohort- und Coach-Dropdown (Coach default null)
- SWR-Hook `useShifts`

DoD: Shifts im UI verwaltbar.

HAUPTFEATURE & ABSCHLUSS

SP-10 ~2,5 Tage

Schedule-View

Ziel: Wochen-Raster mit Zuweisung — das Herzstück der App.

- Wochen-Picker (Mo–Fr); Range → `GET /api/shifts?from=&to=` + `GET /api/users`
- Raster: „Unassigned“-Spur oben, darunter je User eine Coach-Spur
- Client-seitiges Grouping der Shifts nach Coach
- Coach-Dropdown auf der Shift-Karte ⇒ `PUT .../coach` (inkl. Unassign) + `mutate()`

DoD: Zuweisen verschiebt die Karte live von „Unassigned“ in die Coach-Spur.

SP-11 ~1,5 Tage

CD & Deployment (Docker → Docker Hub → Render)

Ziel: Automatisiertes Deployment (idealerweise früh in Woche 1 aufgesetzt, hier gehärtet).

- Multi-Stage- Dockerfile : mvn package → schlankes JRE-Runtime-Image
- GitHub Action: Image bauen → zu **Docker Hub** pushen → **Render-Deploy-Hook** triggern
- Render: **Web Service** (Docker-Image) + managed **PostgreSQL**
- Secrets/Env (OAuth, DB-URL, Docker-Hub-Token, Render-Hook); Coverage/Sonar-Findings aufräumen
- End-to-End-Durchlauf: Login → CRUD → Schedule-Zuweisung auf der deployten App

DoD: Push aktualisiert das Docker-Hub-Image und deployt automatisch auf Render; keine kritischen Findings.

Zeitschätzung & Ausblick

BEREICH	AUFWAND
Fundament (SP-1, SP-2)	~3 T
Auth (SP-3)	~2,5 T
User (SP-4, SP-5)	~3 T
Cohort (SP-6, SP-7)	~2 T
Shift (SP-8, SP-9)	~3,5 T
Schedule (SP-10)	~2,5 T
Deploy & Härtung (SP-11)	~1,5 T
MVP gesamt	~15–18 Dev-Tage (≈ 3 Wochen)

Bei 20 Arbeitstagen in 4 Wochen bleibt nach dem MVP **~1 Woche** für Post-MVP.
Realistisch: **ein größeres Feature oder zwei kleine**. Größte Zeitfresser/Risiken: das OAuth2-Setup (SP-3) und die Schedule-View (SP-10) — ab der zweiten Entität beschleunigt das wiederholte CRUD-Muster.

Post-MVP-Optionen

POST-A ~3–4 Tage

CSV-Import inkl. Feiertage (Flaggschiff)

- CSV je Kurs importieren → Shifts einer Cohorte automatisch anlegen
- Bundeslandspezifische Feiertage via **Nager.Date-API** ausklammern

POST-B ~2 Tage

Off-Times

- Spezielle Shifts, die eine Coach-Zuweisung in diesem Zeitraum blockieren

POST-C ~2 Tage

Rollen-Management

- Rollen jenseits von ADMIN; Coach-Lanes dann rollengefiltert